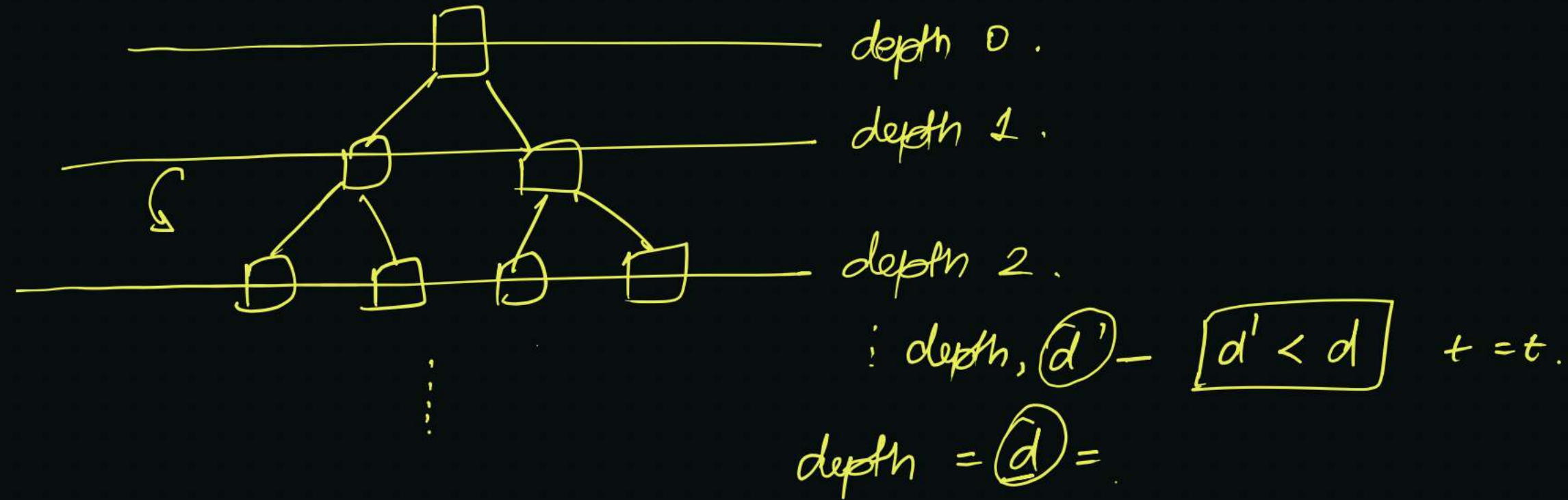


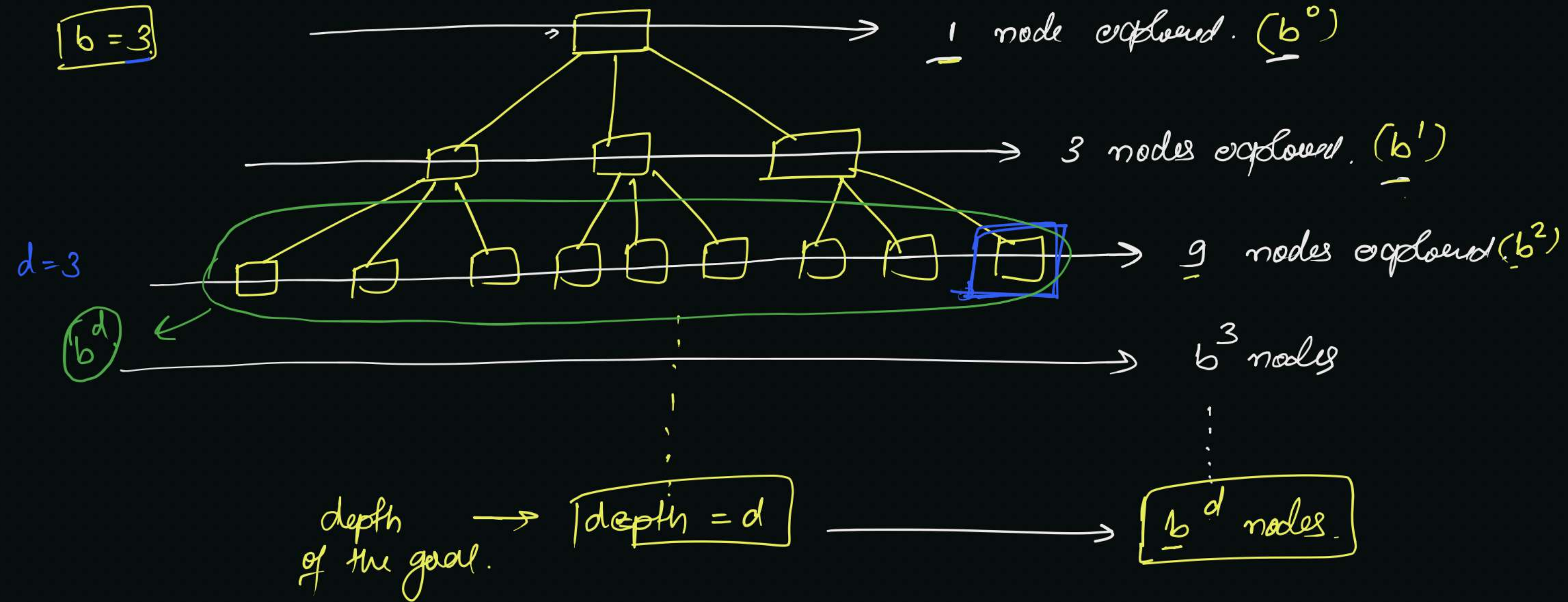
Performance analysis :-

1. Complete :- Yes, it will reach the goal (b is finite, sol. exists).



2. Time :-

↳ Imagine searching a uniform tree where every state has b successors.



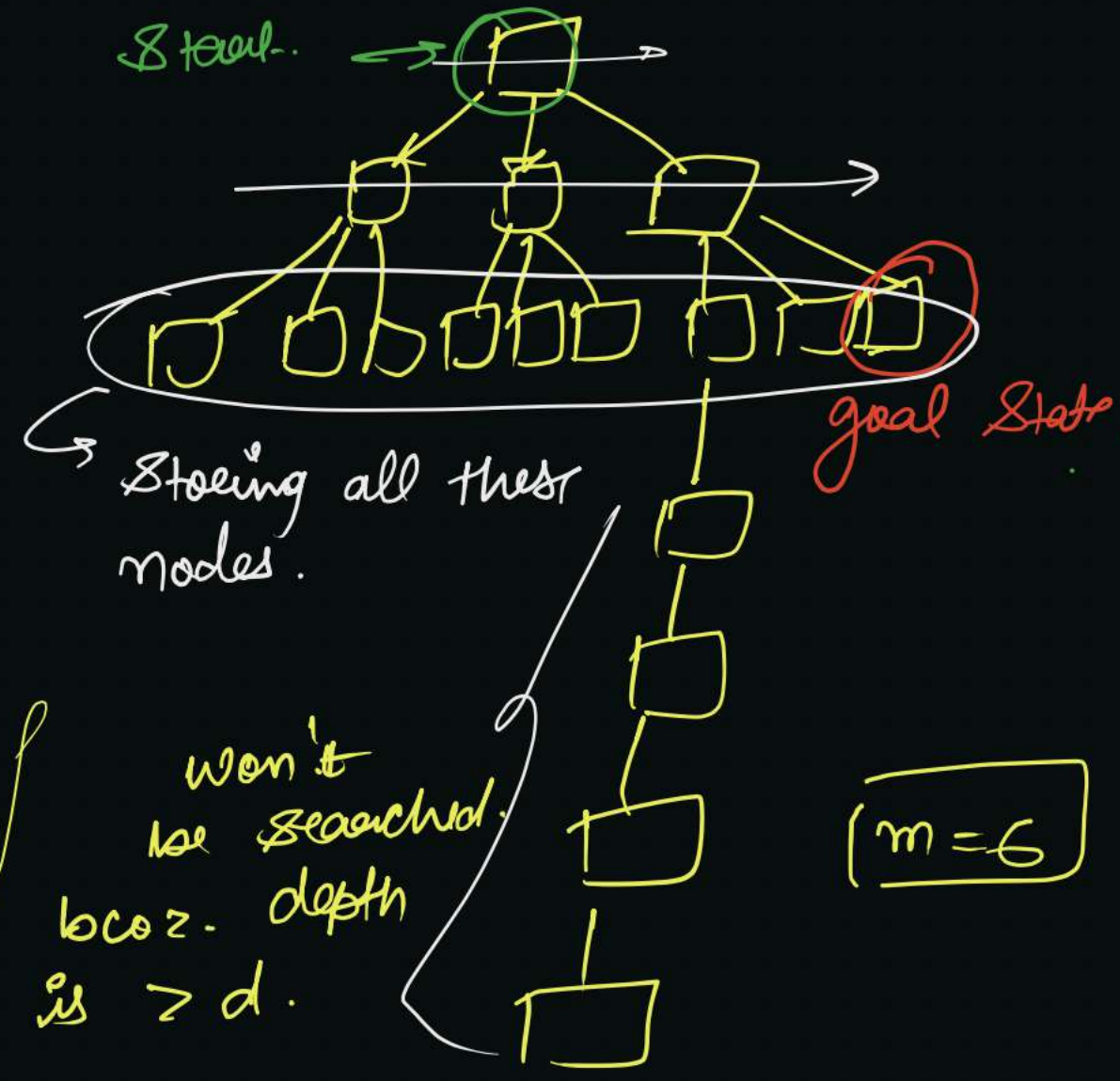
$$\begin{aligned} \text{time complexity} &= b^0 + b^1 + b^2 + b^3 + \dots + b^{d-1} + b^d \\ &= O(b^d) \end{aligned}$$

$b \rightarrow$ max. branching of any node.
 $d \rightarrow$ shallowest goal's depth.

3. Space :-

↳ Worst case, all nodes at depth d will be in memory.
that will take b^d space.

$$\text{Space complexity} = O(b^d)$$



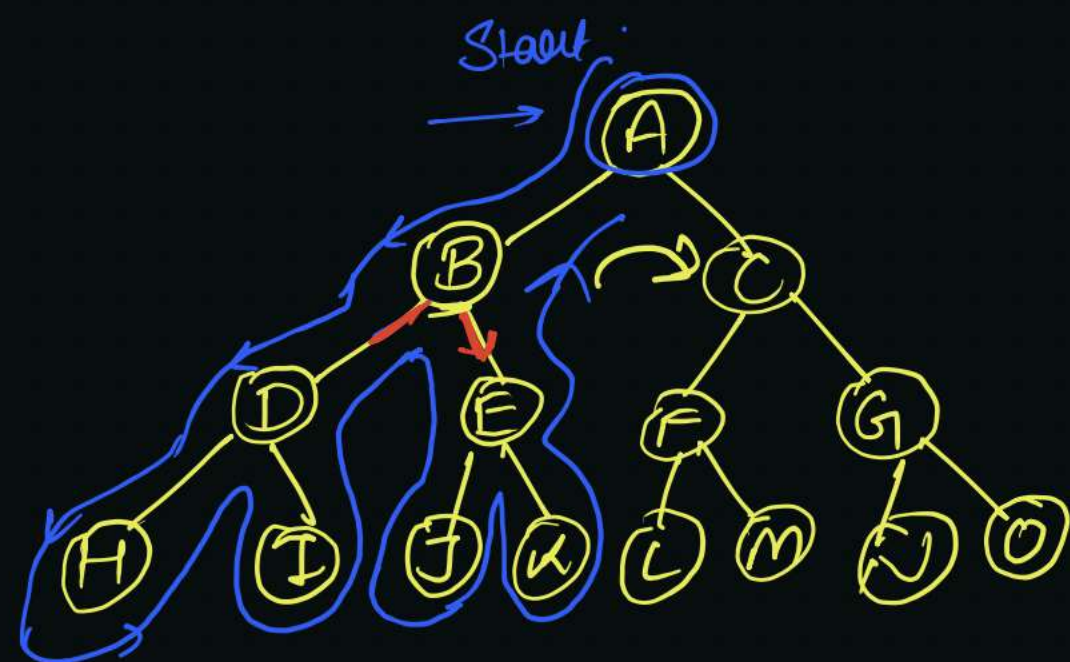
4. Optimal :-

Yes, if step count = 1

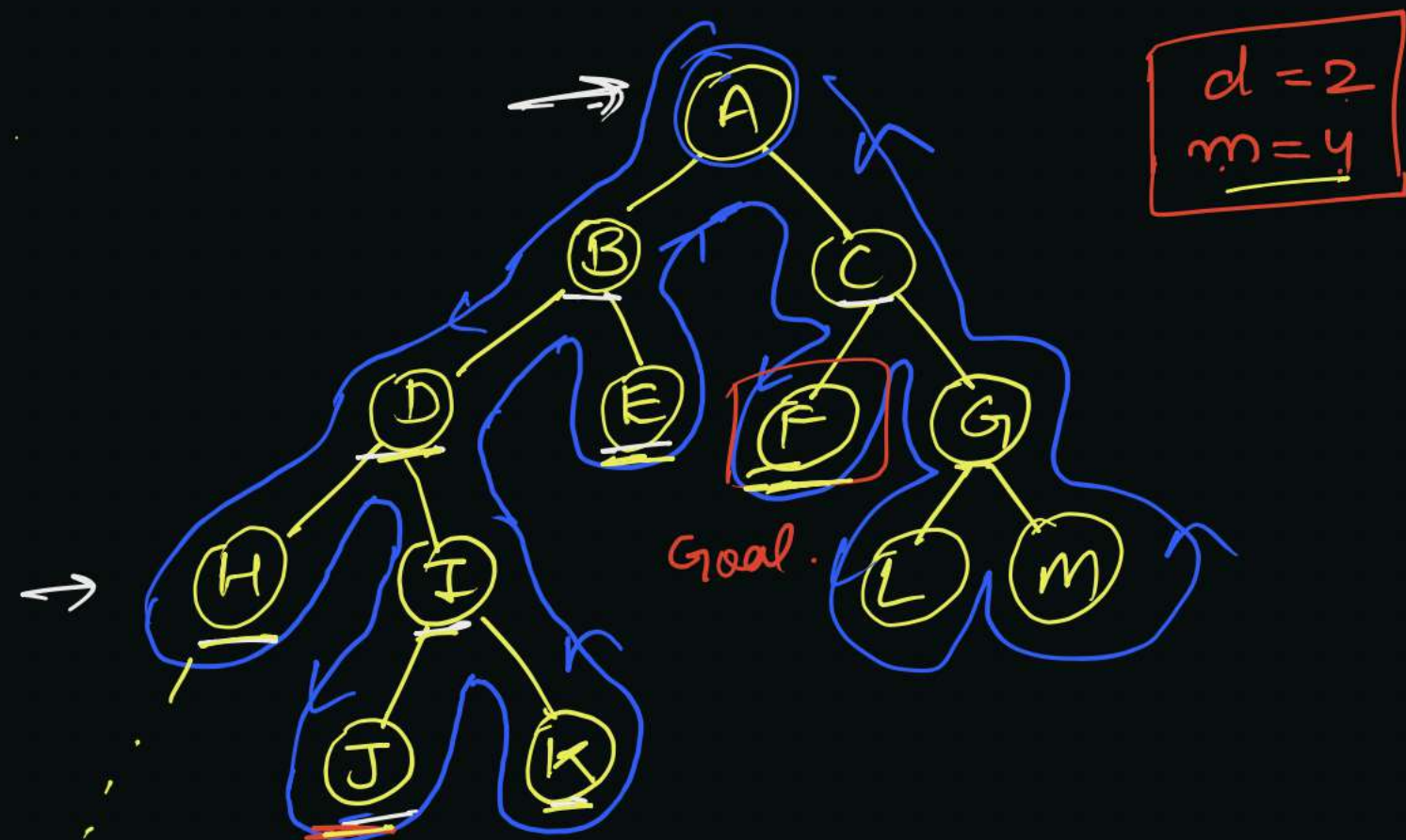
$$d = \text{Cost}$$

② DFS :- Expand the deepest unexpanded node.

eg1 .

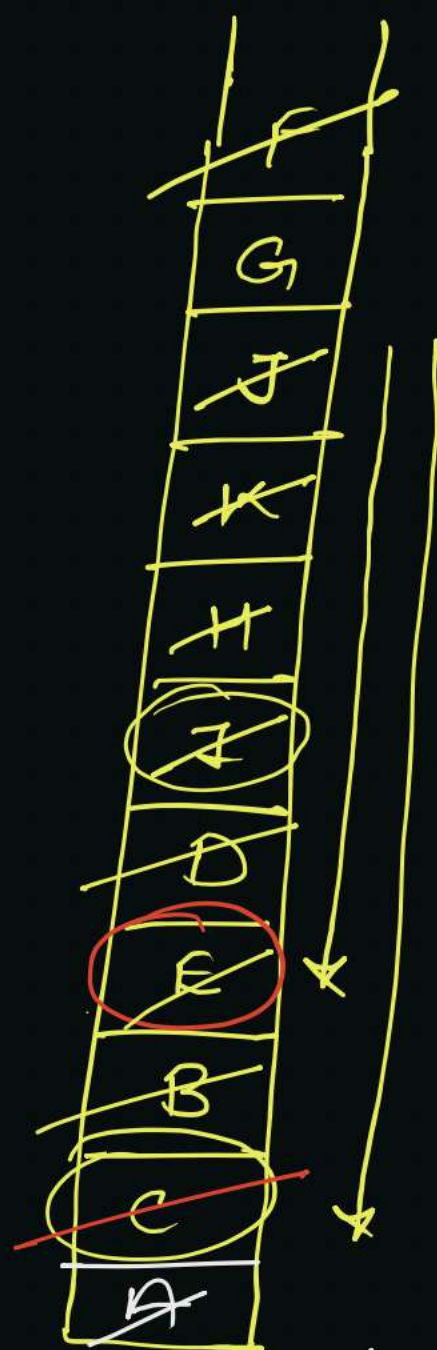


~~eq 2~~


$$\begin{aligned} d &= 2 \\ m &= 4 \end{aligned}$$

A, B, D, H, I, J, K,

E, C, **F**, G, L, M

$$m \rightarrow \infty$$


$C = \{A, B, D, H, I, J, K, E, C, F\}$

we reached
the goal.

[illegible]

Pseudo algorithm :-

take s as an empty stack

$s.push(start)$

mark $start$ as visited.

while (s is not empty)

{

→ $t = s.pop()$

if ($t = g$) → return t ; → goal state is achieved.

else

{

→ for all neighbours t' of t

{

if (t' is not visited)

{

$s.push(t')$

mark t' as visited.

}

}

}

}

$s \rightarrow$ stack

$start \rightarrow$ starting/initial node.

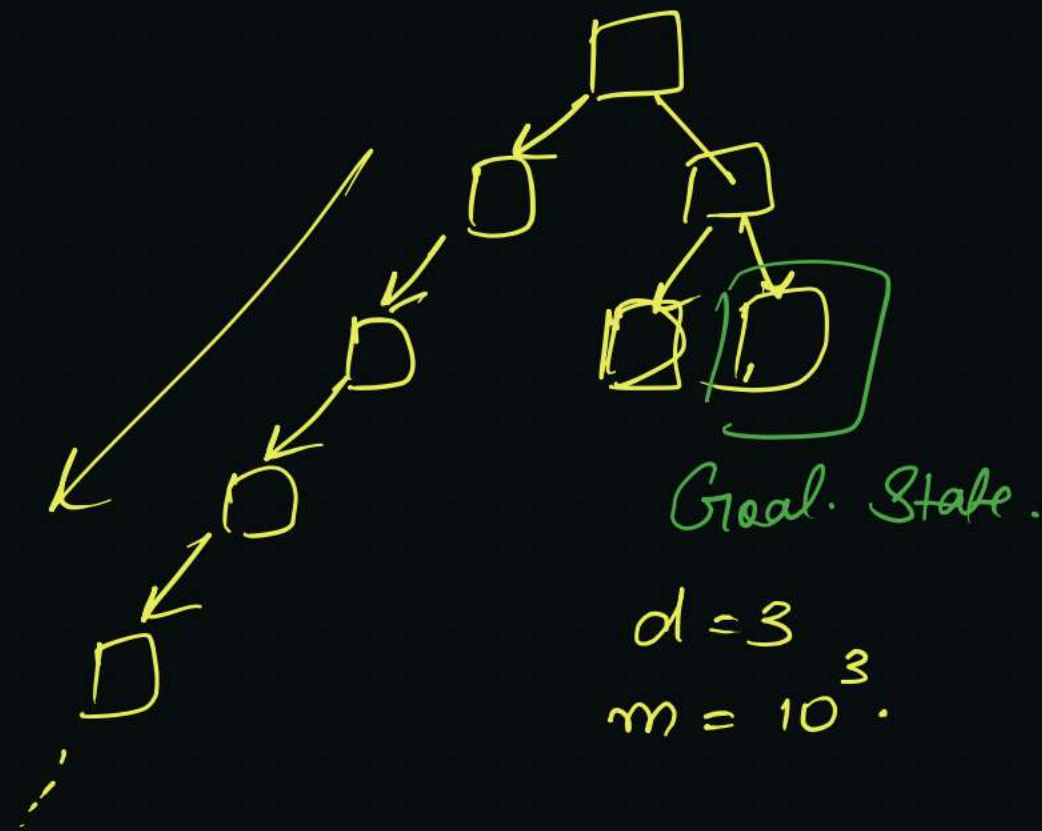
$g \rightarrow$ goal state.

Performance analysis :-

1. Complete :- No, maximum depth (m) can be infinite.
 $m \rightarrow \infty$

2. Time :-

$$\text{Time complexity} = O(b^m)$$

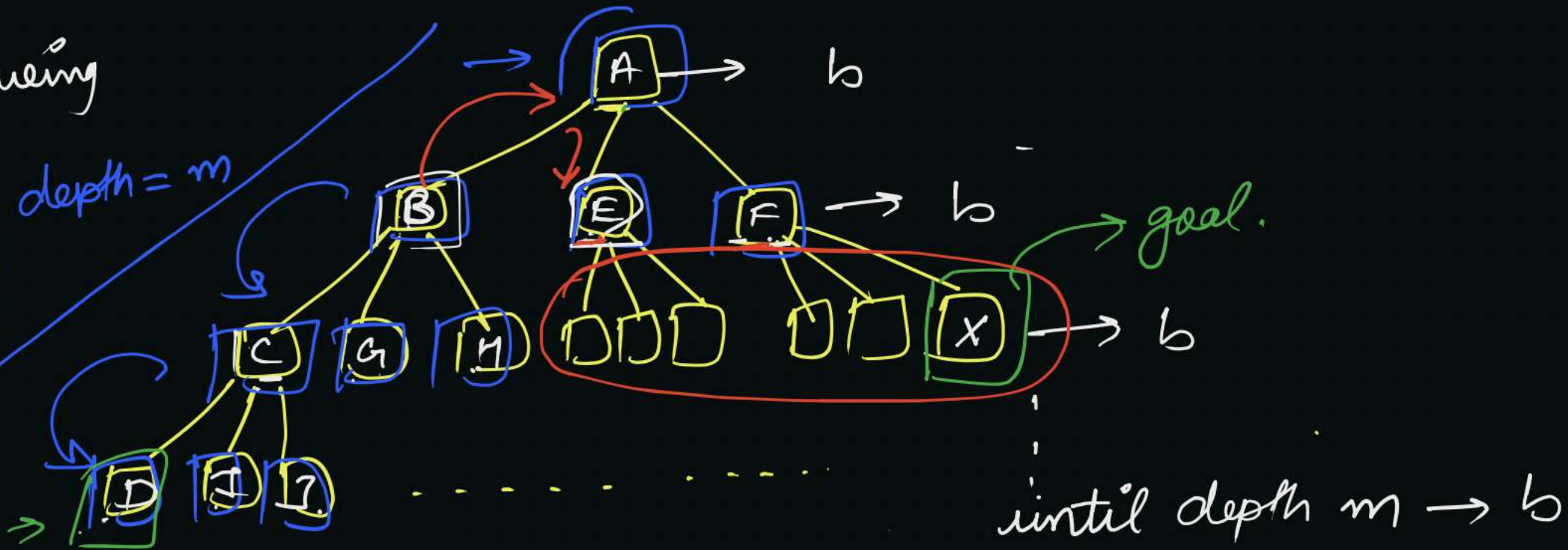


3. Space :- Each node having b successors.

$$\text{Space complexity} = O(bm)$$

each node in the tree having b children

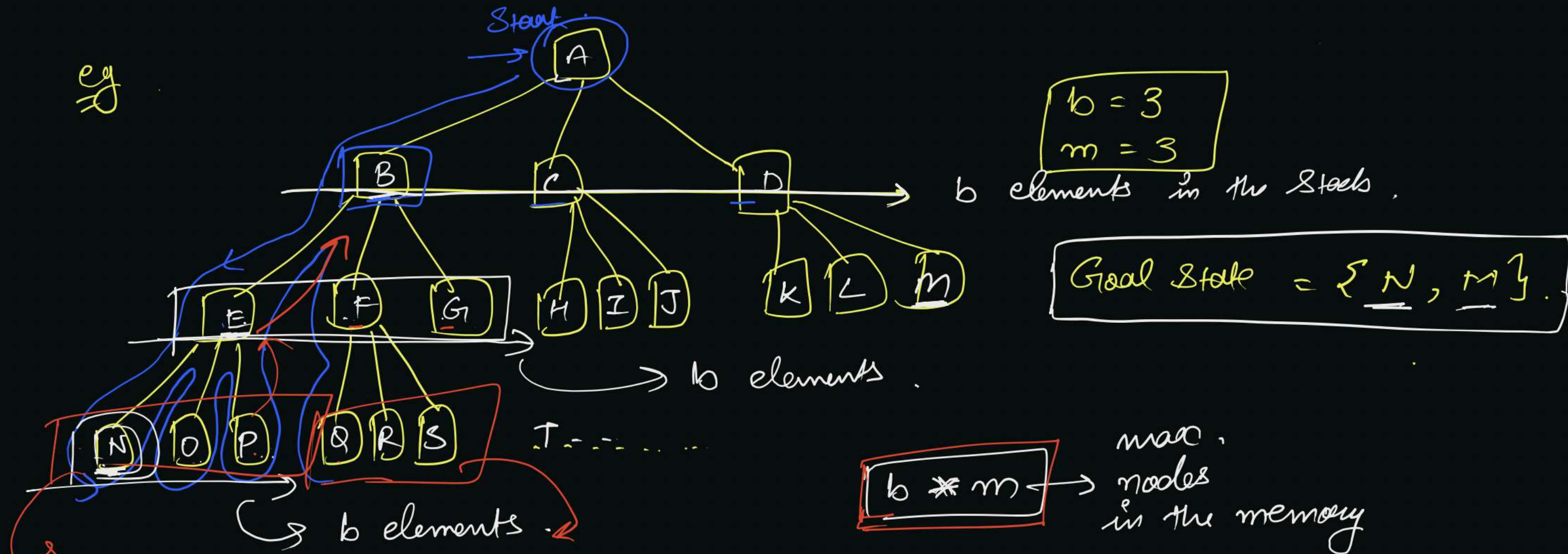
max depth.



$A \rightarrow B \rightarrow C \rightarrow D$

$A, B, E, F, C, G, H, D, I, J,$

eg



(b) N, O, P are released & Q, R, S (b) need b nodes will be added.

4. Optimal :- Not optimal, returns the first solution it finds. Even if it's not the cheapest.

eg. In the above example, Goal State = {M, N}

DFS will return (N) → as the solution, which is not optimal

Cost = Step Count = 1.

Cost (N) = 3 → DFS will return this one.

Cost (M) = 2 → most optimal solution.